INTERNSHIP-III REPORT

On

Analysis and Implementation of FFT Algorithms

Submitted by

ROOPESH O R (20323085)

in partial fulfillment of the requirements for the award of the degree of
B.Tech in Electronics and Communication Engineering



Internship carried out at

Advanced System Laboratory - DRDO, Hyderabad

Under the Guidance of

G. Venkat Reddy, Sc 'F'

Internship Duration:

May 18 2026 - July 15 2026

DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI - 682022

July 2026

DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI-682022

CERTIFICATE

Certified that the internship report titled ***"ANALYSIS AND IMPLEMENTATION OF FFT ALGORITHMS"*** *is a bonafide work of* ***ROOPESH O R*** *towards the partial fulfillment for the award of the degree of B.Tech in Electronics and Communication of Cochin University of Science and Technology, Kochi-682022.*

Internship Coordinator **Head of the Division**

Acknowledgment

I would like to express my heartfelt gratitude to the Advanced Systems Laboratory, Defence Research and Development Organization for giving me the opportunity to undertake my internship. Special thanks to G. Venkat Reddy, Sc ‘F’, ASL DRDO, for selecting and guiding me through this internship. We also thank our professors for their constant support and inspiration. Special thanks to our HoD, Dr. Deepa Shankar and our department faculty, Dr. Mridula S. Furthermore, I thank my family and my friends who motivated me in this journey.

Table of contents

List of Figures ..... **ii**

List of Tables ..... **iii**

1. About the Organisation **1**

2. Overview of internship **2**

2.1. Introduction 2

2.2. Artix-7 2

3. Techniques **3**

3.1. Introduction 3

3.2. Method used in analysis 3

3.3. Cooley-Tukey: Radix-2 DIT 4

3.3.1. Implementation 6

3.3.2. Simulation result 7

3.4. Radix-2 Multi Path Delay Commutator (R2MDC) 8

3.4.1. Implementation 9

3.4.2. Simulation result 10

3.5. Radix-2 Single Path Delay Commutator (R2SDC) 11

3.5.1. Implementation 11

3.5.2. Simulation result 13

3.6. Winograd 13

3.6.1. Derivation for 5 point case 13

3.6.2. 16 Point Winograd algorithm 17

4. Comparison **19**

4.1. Comparison of 16 point transforms 19

4.2. Comparison of scalable techniques 19

5. Conclusion **21**

6. Future scope **22**

References **23**

Appendices **24**

A.1 Source code 24

A.2 16 point Winograd algorithm 24

List of Figures

Figure 3.1 Steps involved in Cooley-Tukey algorithm 4
Figure 3.2 Signal flow graph of 8-point radix-2 DIT-FFT 5
Figure 3.3 Circuit of Radix-2 In-place implementation 6
Figure 3.4 Working of FSM in Radix-2 DIT FFT 7
Figure 3.5 Simulation result of 4 point Radix-2 DIT-FFT for a ramp input 8
Figure 3.6 Signal flow graph of 4-point radix-2 DIF-FFT 8
Figure 3.7 Steps in R2MDC algorithm for 4 point case 9
Figure 3.8 Schematic of R2MDC implementation 10
Figure 3.9 Inside a single R2MDC Stage. All modules with wedge is clocked 10
Figure 3.10 Simulation result for two 4 point test signals in R2MDC 10
Figure 3.11 Schematic of R2SDC implementation 11
Figure 3.12 Inside a single R2SDC Stage. All modules with wedge is clocked 11
Figure 3.13 Steps involved in a 4 point R2SDC algorithm 12
Figure 3.14 Simulation result for 4 point complex test signal in R2SDC 13
Figure 3.15 Signal flow of modified Winograd algorithm for 5 point DFT. 17
Figure 3.16 Output of 16 point 4-stage pipelined Winograd implementation for 2 consecutive inputs. 18
Figure 3.17 Comparison of output of FFT in MATLAB and simulation 18
Figure 4.1 Resource and performance analysis of scalable techniques. 20

List of Tables

Table 3.1 Polynomial coefficients in winograd algorithm 15
Table 4.1 Comparison of various 16 point techniques 19

Chapter 1:
About the Organisation

The Defence Research and Development Organization (DRDO) is India's premier agency under the Ministry of Defence, responsible for the research and development of state-of-the-art defence technologies. Established in 1958, DRDO was formed by merging the Technical Development Establishment of the Indian Army and the Directorate of Technical Development and Production with the Defence Science Organization.

With a mission to empower India with cutting-edge defence technologies and reduce dependency on foreign imports, DRDO has grown into a network of over 50 laboratories, specializing in various disciplines such as aeronautics, armaments, electronics, missiles, combat vehicles, naval systems, advanced computing, simulation, and life sciences.

DRDO plays a crucial role in the development of strategic and tactical systems, including the Agni and Prithvi missile series, Tejas Light Combat Aircraft, Arjun Main Battle Tank, electronic warfare systems, unmanned aerial vehicles, and many more. These innovations not only enhance the strength of the Indian Armed Forces but also contribute to self-reliance through the "Make in India" initiative.

One of DRDO's key strengths lies in its world-class scientists and engineers, who work tirelessly across domains to address the unique challenges faced by the nation. Its Advanced System Laboratory (ASL), where this internship was conducted, is a leading laboratory engaged in the development of solid propulsion technologies, composites, aerospace mechanisms, etc.

DRDO is committed to continuous innovation, collaboration with academia and industry, and the pursuit of excellence in all its endeavors. By combining technological advancement with national security priorities, DRDO continues to be a cornerstone in India's journey towards defence self-sufficiency and technological leadership

Chapter 2:
Overview of internship

2.1 Introduction

In Digital Signal Processing (DSP), the need to express a time domain signal in the frequency domain arises in various problems. For this, the Fourier Transform (FT) is widely used. However FT is computationally expensive. In past century various techniques have been proposed to compute FT of signals with limited compute power and memory. These set of techniques are now commonly called Fast Fourier Transform (FFT) algorithms.

My internship involves understanding, analyzing and implementing few of these algorithms using SystemVerilog in the context of FPGA. The architecture, derivation and operation of these algorithms are detailed in the following chapters. Mainly 4 algorithms are examined, namely: Inplace, R2MDC, R2SDC and Winograd. The hardware requirements and performance aspects are also discussed in *Section 4: Comparison*. In the summary, an overall analysis of all algorithms are outlined.

For analysis, Artix-7 family of FPGA was targeted in all the implementation. Artix-7 was chosen as I found it frequently appearing in various papers. Operating frequency of 100MHz was targeted in all algorithms. However as we will see that some algorithm cannot achieve this while others can go beyond this point.

2.2 Artix-7

Artix-7 is a family of cost-optimized FPGAs developed by AMD suitable for low power applications requiring serial transceivers and high DSP and logic throughput while providing the lowest total bill of materials. Built on state-of-the-art, high-performance, low-power (HPL), 28 nm, high-k metal gate (HKMG) with 1.0V core voltage process technology and 0.9V core voltage option for even lower power. The specific part number used is `xc7a100tcsg324-1`. Its salient features are: [1]

- 15,850 CLB slices (each containing four 6-input LUTs and eight flip-flops. Some slices can use their LUTs as distributed RAM)
- 4860 Kb dual-port block RAM
- 240 DSP slices with 25 x 18 multiplier, 48-bit accumulator, and pre-adder [2]
- 101,440 Logic cells
- 13 Mb Block RAM
- 300 User I/O with 1.2V – 3.3V voltage rating

Chapter 3:
Techniques

3.1 Introduction

First 3 techniques described in this chapter are scalable – that is transform size is parameterized; the desired length DFT computer can be easily generated by tweaking one or two variables. Moreover R2MDC and R2SDC are more practical compared to in-place algorithms. These 3 methods are radix-2 only.

The last algorithm, Winograd is not scalable and has different implementation for different transform sizes. Winograd is often used in conjunction with Cooley-Tukey decomposition, where performance of smaller DFT has significant contribution in overall performance [3]. For Winograd, a scheme for 5 point DFT is derived, at the end evaluation of 16 point Winograd is given.

3.2 Method used in analysis

The initial development, simulation and debugging was done using `Verilator` due to its ease of use. Later, synthesis and implementation were carried out in Vivado. To automate the process of generating implementation reports for multiple point size FFTs in the case of scalable techniques, a TCL script was created and Vivado was run from terminal using the script. The reports such as hardware utilisation, nets, routing summary, timing and power characteristics were obtained using these commands respectively:

```
report_utilization -hierarchical
report_route_status -show_all
report_timing_summary
report_power
```

For estimation of timing, a 10ns clock was assumed. And for power utilisation the toggle rate and static probability was prescribed, (hence doing a vectorless analysis). Input control signals were given toggle rate of 2% and while for inputs 50%. static probability is 95% for input control signals and 50% for data inputs

```
set_switching_activity -toggle_rate 2.0 -static_probability 0.95 [get_ports in_valid]
set_switching_activity -toggle_rate 50.0 -static_probability 0.5 [get_ports {din[*]}]
set_switching_activity -toggle_rate 0.0 -static_probability 0.0 [get_ports rst]
```

With these constraints, tool reports `medium` confidence in power estimation. A measure of "latency" is discussed throughout this report which is implied as the time or number of clock cycles required to obtain first output after the first input has been passed. Throughput is the

number of samples the algorithm can process in a cycle. Since here I have included both pipelined and non-pipelined implementations, throughout is defined as:

$$\frac{\text{Number of samples}}{\text{Number of cycles to process whole input}} \times \text{max frequency of operation}$$

3.3 Cooley-Tukey: Radix-2 DIT

Cooley-Tukey algorithm re-expresses the Discrete Fourier Transform (DFT) of an arbitrary composite size $N = N_1 N_2$ in terms of N_1 smaller DFTs of sizes N_2, recursively, to reduce the computation time to $\mathcal{O}(N \log N)$ for highly composite N. Because of the algorithm's importance, specific variants and implementation styles have become known by their own names, such as Radix-2, Radix-4, mixed-radix, etc. Because the Cooley-Tukey algorithm breaks the DFT into smaller DFTs, it can be combined with any fast implementation of smaller size DFTs.

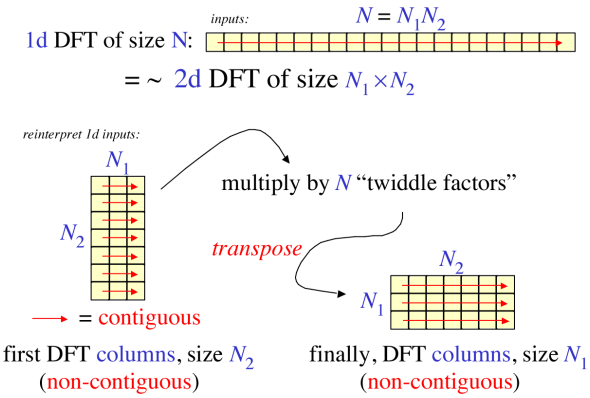


Figure 3.1: Steps involved in Cooley-Tukey algorithm

A radix-2 Decimation in Time (DIT) FFT is the simplest and most common form of the Cooley-Tukey algorithm. Radix-2 DIT divides a DFT of size N into two interleaved DFTs of size $N/2$ with each recursive stage. Hence the overall time complexity of algorithm is $\mathcal{O}(N \log N)$. The following is the derivation of radix-2 DIT:

DFT of sequence x_n is defined by:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-j\frac{2\pi}{N}nk\right)$$

The radix-2 DIT algorithm rearranges the DFT of the function x_n into two parts: a sum over the even-numbered indices $n = 2m$ and a sum over the odd-numbered indices $n = 2m + 1$:

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j\frac{2\pi}{N}(2m)k\right) + \sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j\frac{2\pi}{N}(2m+1)k\right)$$

$$X_k = \underbrace{\sum_{m=0}^{N/2-1} x_{2m} \exp\left(-j\frac{2\pi}{\frac{N}{2}}mk\right)}_{\text{DFT of even-indexed part of } x_n} + \exp\left(-j\frac{2\pi}{N}k\right) \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} \exp\left(-j\frac{2\pi}{\frac{N}{2}}mk\right)}_{\text{DFT of odd-indexed part of } x_n}$$

The even and odd parts can be written as:

$$X_k = E_k + \exp\left(-j\frac{2\pi}{N}k\right)O_k$$

Using the periodicity of the complex exponential, $X_{k+\frac{N}{2}}$ is also obtained from E_k and O_k as:

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right)O_k$$

We can write X_k and $X_{k+\frac{N}{2}}$ as:

$$X_k = E_k + \exp\left(-j\frac{2\pi}{N}k\right)O_k$$

$$X_{k+\frac{N}{2}} = E_k - \exp\left(-j\frac{2\pi}{N}k\right)O_k$$

This result, expressing the DFT of length N recursively in terms of two DFTs of size N/2, is the core of the radix-2 DIT FFT. The algorithm gains its speed by re-using the results of intermediate computations to compute multiple DFT outputs. The final outputs are obtained by combination of E_k, O_k and $\exp(-j2\pi k/N)$ (called "twiddle factor" or phase rotation factor) which is simply a size-2 DFT (also called "Butterfly"). This recursive splitting of terms into even and odd terms results in shuffling of input terms, while preserving the order of output. The operation can be expressed as a signal flow graph shown in Figure 3.2. [4]

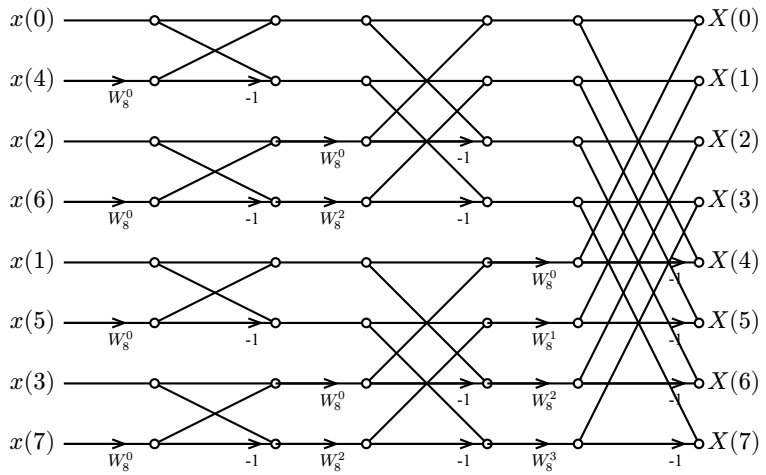


Figure 3.2: Signal flow graph of 8-point radix-2 DIT-FFT

In the software, assuming we have array of input x, the computation begins with reordering the array by swapping element at index i with element at an index which is bit reverse of i [5]. Then subsequently performing radix-2 recursively. The advantage of hardware design (in some algorithms) is that the reordering can be done with appropriate wiring wiring, hence no need of a separate reordering stage. The technique outlayed in this subsection, however, does not takes advantage of this property, due to the very nature of implementation.

3.3.1 Implementation

The implementation accepts a *fixed point* complex number (given through 2 separate buses for real and imaginary part) and returns computed DFT through another 2 separate buses. This is a burst architecture, hence it waits for all inputs to arrive before starting the computation. Also next inputs can be given to the module only after it finishes previous computation.

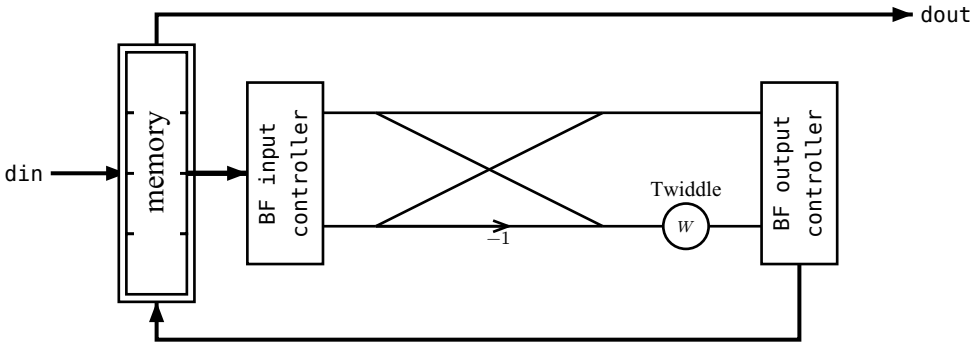


Figure 3.3: Circuit of Radix-2 In-place implementation

The operation described below:

1. A reset pulse is applied
2. Start signal is issued which starts a counter inside the module.
3. `input_valid` signal is applied and input is passed in each positive edge of clock.
4. After N clock cycle, when all input are be given, `input_valid` is reset and module starts computing
5. After $N \log_2(N)/2$ clock cycles the output will be available
6. To read output, output read enable is raised and read address is passed. In the next clock cycle the output will be obtained.

Later designs described in this chapter follows similar operation. The whole process takes $N + N \log_2(N)/2 + 1$ clock cycles from passing first input to getting first output: N cycles for applying input and getting output and $N \log_2(N)/2 + 1$ cycles for computation.

The design is fully parametric – the number of points and data width can be changed. The twiddle factor is generated using a python script. Design is limited in terms of throughput and latency and is not useful for high performance use cases. In theory, this should be most area

efficient design, however due to multiple access to the memory, the synthesizer couldn't infer a BRAM, instead it went for flip-flops. As a result the design is not scalable to a very large extent.

The implementation is by no means efficient for practical use, but serves as a base upon which other algorithms can be implemented and evaluated.

About FSM:

When a start signal is sent, the internal counter starts running. Based on the value of the counter, the addresses of relevant data – inputs to butterfly and twiddle factor – is generated by address generator. A portion of the counter is bit reversed to produce an address that will be used to store the input. The computing modules and FSM all also run in parallel. The FSM has 4 states:

- IDLE (not doing any processing)
- LOAD(loading input to internal memory)
- COMPUTE (computing the results and storing it in the memory)
- DONE: All computations are done and ready for next cycle.

The operation of FSM is summarized in Figure 3.4. After the ouput is ready, the can be read by raising `output_read_en` and passing the address to be read in `output_read_addr`.

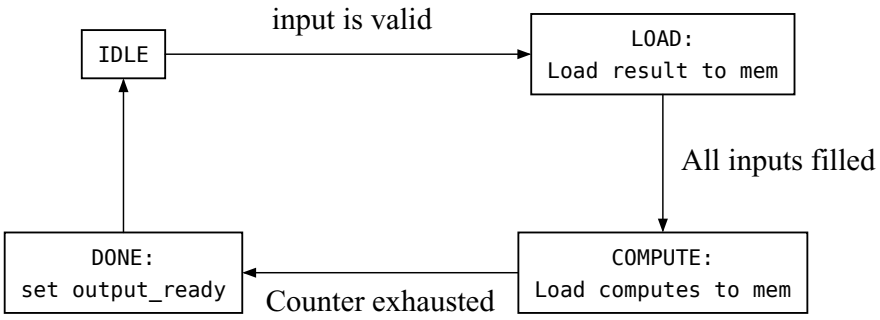


Figure 3.4: Working of FSM in Radix-2 DIT FFT

3.3.2 Simulation result

Here 4 point DFT is computed in which input is real valued ramp signal. In the Figure 3.5, the output is observed at `dout_re` and `dout_im`. The results were verified using MATLAB. From this we can notice that there is a 9 clock cycle ($N + \frac{N}{2}\log_2(N) + 1$) latency to get first output (one additional clock cycle for shift register at output)

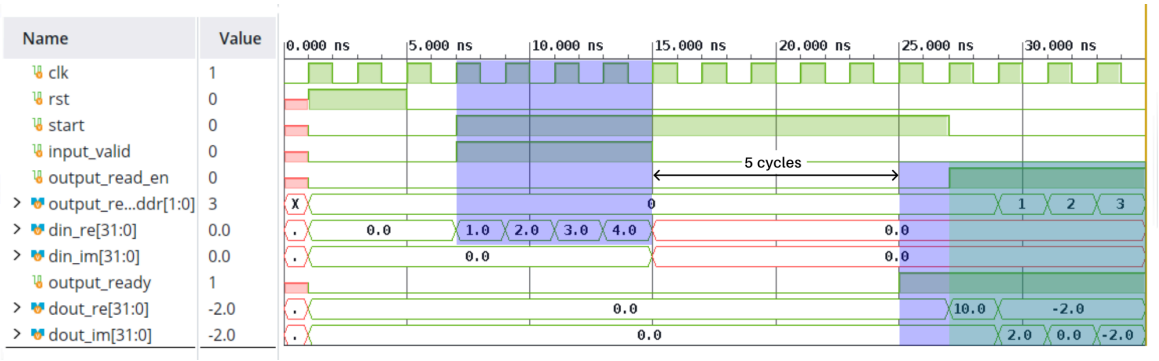


Figure 3.5: Simulation result of 4 point Radix-2 DIT-FFT for a ramp input

3.4 Radix-2 Multi Path Delay Commutator (R2MDC)

R2MDC is based on the Decimation in Frequency (DIF) method. In DIF, the inputs are applied in order, while output obtained will be in bit-reversed order. This allows the input to be fed sequentially. The signal flow graph of DIF is inverted version of DIT. However the final output has particular pattern that can be used to transform this parallel structure to a pipelined multistaged architecture. [6]

The usual structure of DIF in the case where $N = 4$ is shown in Figure 3.6. Notice that in stage 0, kth and $\left(k + \frac{N}{2}\right)$th inputs are required ($x_0, x_2$ and x_1, x_3). If the data is fed sequentially then one can store first 2 data into a shift register and pop out the value afterwards. This gives the input required for butterfly in the next stage. So the corresponding outputs in each stages are: x'_0, x'_2 and x'_1, x'_3. But the next stage needs pairs x'_0, x'_1 and x'_2, x'_3. So twiddle multiplied output of this stage needs to be stored for 1 cycle. This will satisfy the condition for next stage.

In general at stage $s(= 0, 1, ...S$ where $S = \lceil \log_2(N) \rceil$ = number of stages) needs shift register of size 2^{S-s} in upper input arm of butterfly and 2^{S-s-1} shift register after the twiddle path. For the last stage, multipliers and commuters are not needed For higher point FFT, design can be scaled easily by cascading more and more R2MDC stages [7].

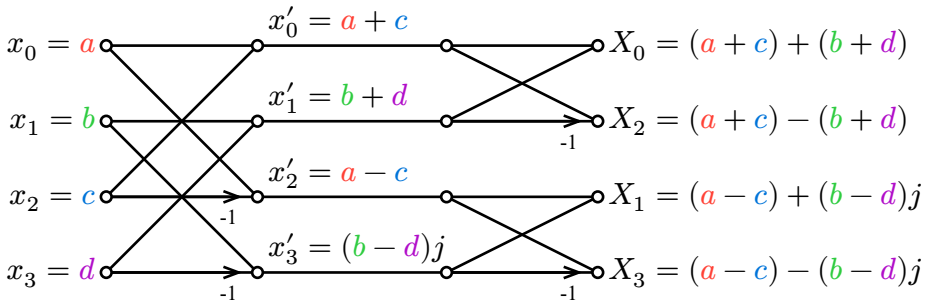


Figure 3.6: Signal flow graph of 4-point radix-2 DIF-FFT

3.4.1 Implementation

The design is fully parametric, in which number of stages, and data width can be changed The design consists of a controller block that starts the global counter and produces control signals. The outputs `out0` and `out1` are passed to cascade of R2MDC Stage blocks as shown in Figure 3.8. Each R2MDC stage is a pipelined design consisting of a butterfly unit, twiddle ROM and commutator. Butterfly here only computes sum and difference of incoming inputs, the multiplication with twiddle factor is done using a pipelined multiplier. The required twiddle factors are computed during synthesis and are stored in a LUT. The control signals are also delayed by the same amount as that of data lines.

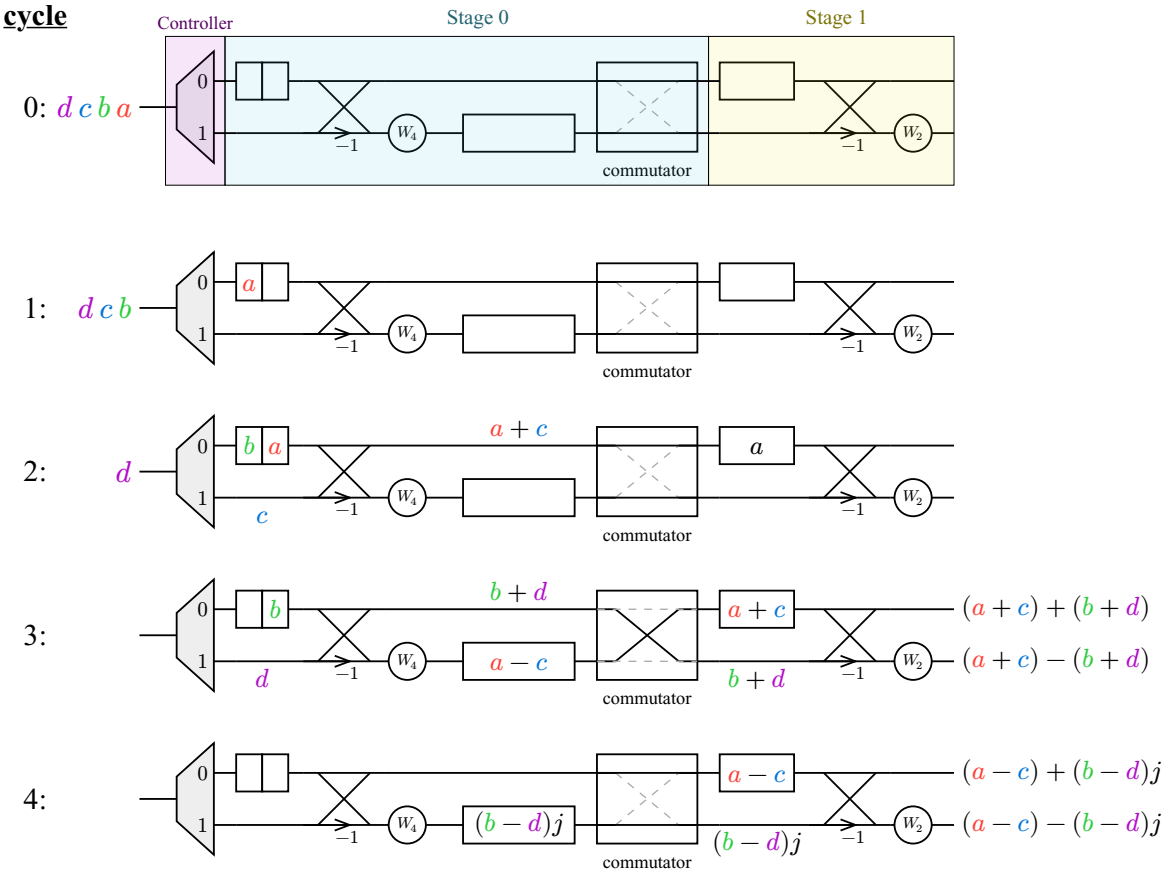


Figure 3.7: Steps in R2MDC algorithm for 4 point case

In each stage the size of delay element reduces by half. No delays, multipliers and commutators are required in final stage. The final output is obtained at `out0` and `out1` corresponding to bit reversed indexed $X[k]$ and $X[k + N/2]$. The control signals `valid` and `counter` are synchronously passed from one stage to another, using delay. As shown in Figure 3.9, two unit delays are used for pipelining: one for splitting addition and multiplication in butterfly and another one for pipelining complex multiplication in multiplier.

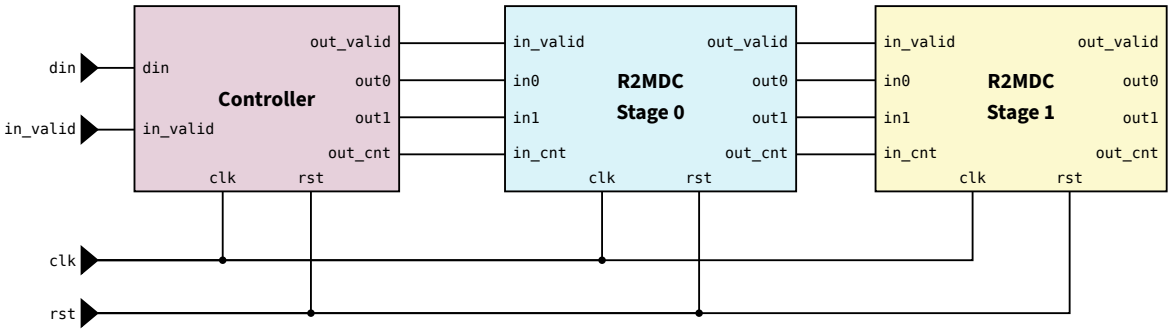


Figure 3.8: Schematic of R2MDC implementation

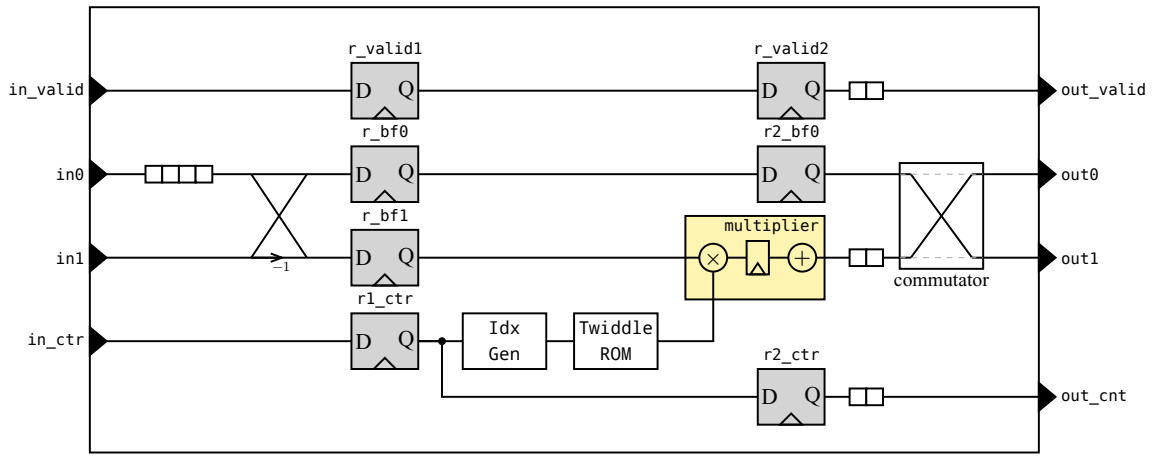


Figure 3.9: Inside a single R2MDC Stage. All modules with wedge is clocked

3.4.2 Simulation result

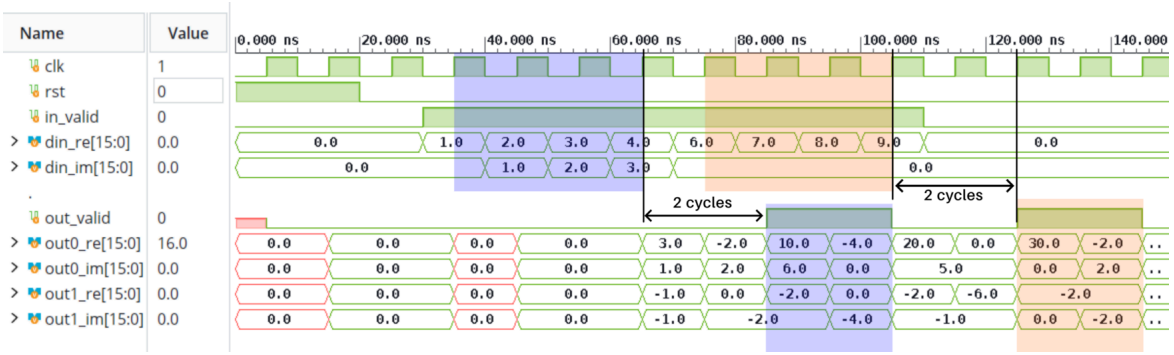


Figure 3.10: Simulation result for two 4 point test signals in R2MDC

Here 2 different inputs are passed consecutively, marked by different colors. For each sequence, 2 outputs are obtained at same clock cycle. It takes $\log_2(N)$ clock cycles to get all ouputs after last input has been inserted, giving a total latency of $N + \log_2(N)$

3.5 Radix-2 Single Path Delay Commutator (R2SDC)

Single path Delay Commutator is a different pipelined implementation of DIF algorithm. Unlike MDC, SDC produces single output per cycle and uses fewer number of shift registers, at the expense of higher latency and higher complexity of stage controllers [7], [8] . The whole operation is described in Figure 3.13. An overview is given below:

1. In first N/2 clock cycles data coming is shifted to shift registers
2. In coming cycles, $x[n]$ and $x[n + N/2]$ datas are available. The butterfly computes the sum and difference, in which the difference is shifted back to the same shift register and sum is passed to twiddle factor multiplier.
3. After all inputs are passed (cycles > N) the shift register is completely full of differences. It shifts it out to twiddle factor directly.

3.5.1 Implementation

The design is similar to that of MDC, with the difference that each stage now has its own counter that creates control signal for that stage.

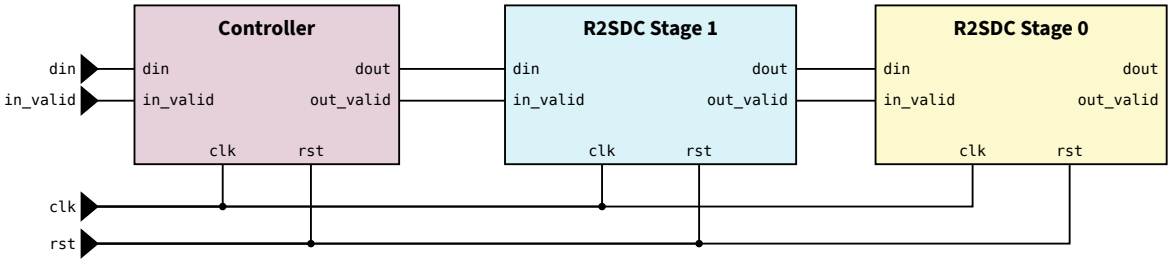


Figure 3.11: Schematic of R2SDC implementation

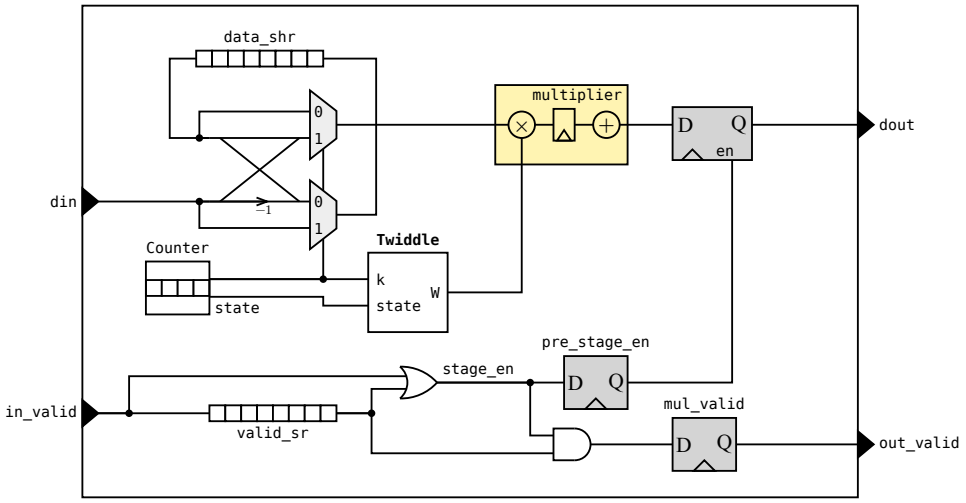


Figure 3.12: Inside a single R2SDC Stage. All modules with wedge is clocked

From this point onwards, the twiddle factors are computed using `initial` blocks, which is expected to be synthesized to a LUT.

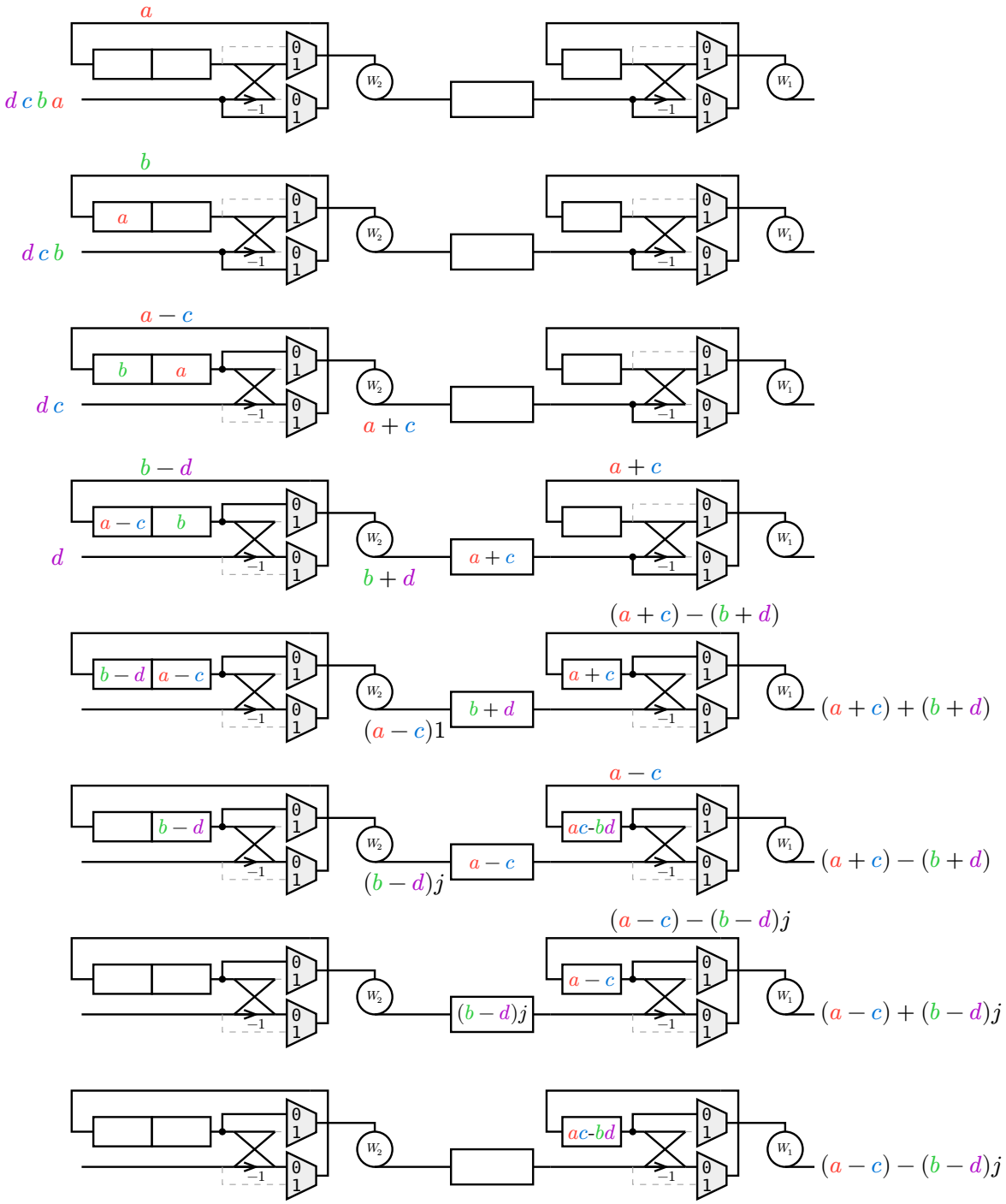


Figure 3.13: Steps involved in a 4 point R2SDC algorithm

3.5.2 Simulation result

The result is similar to that of MDC case. But here due slightly different architecture, the output arrives one clock cycle earlier.

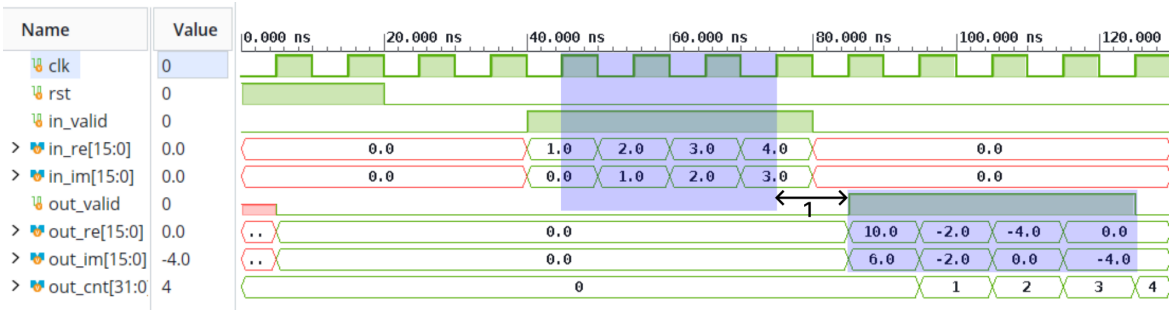


Figure 3.14: Simulation result for 4 point complex test signal in R2SDC

3.6 Winograd

Winograd algorithm is a fast convolution algorithm which reduces number of multiplications (a strong operation) at the expense of increased number of additions (weak operation). The area and time requirement for a multiplier is relatively higher than adder. Hence for large number of bits ($\geq$ 16), this tradeoff results in less silicon area and power. This method also used in the context of Application Specific Integrated Circuit (ASIC) designing [3].

Winograd showed that a prime length DFT can be done as circular convolution, applying input and output Rader index mapping [9]>, supplement: [p. 264])[3]. The method can be derived for any prime size DFT, however the implementation details vary for different lengths. Here implementation of length 5 DFT is shown.

3.6.1 Derivation for 5 point case

The forward transform for 5 point DFT is given by:

$$Y_k = \sum_{n=0}^{4} x_n W^{nk}$$

where $W^{nk} = e^{-j2\pi nk/5}$. Because W represents roots of unity of 5, the exponent value nk wraps around every 5 cycle: ie. $W^{nk+5m} = W^{nk}$ for some arbitrary integer m. The operation can be expressed as a matrix multiplication:

$$\begin{bmatrix} Y(0) \\ Y(1) \\ Y(2) \\ Y(3) \\ Y(4) \end{bmatrix} = \begin{matrix} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} \begin{matrix} \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{bmatrix} W^0 & W^0 & W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 & W^3 & W^4 \\ W^0 & W^2 & W^4 & W^1 & W^3 \\ W^0 & W^3 & W^1 & W^4 & W^2 \\ W^0 & W^4 & W^3 & W^2 & W^1 \end{bmatrix} \end{matrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

The first row corresponds to DC sum, and the rest of elements can be written as:

$$Y_k = x_0 + Y'(k), \quad k > 0$$

Where $Y'(k)$ is

$$\begin{bmatrix} Y'(1) \\ Y'(2) \\ Y'(3) \\ Y'(4) \end{bmatrix} = \begin{bmatrix} W^1 & W^2 & W^3 & W^4 \\ W^2 & W^4 & W^1 & W^3 \\ W^3 & W^1 & W^4 & W^2 \\ W^4 & W^3 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

The matrix operation can be rearranged in such a way that the matrix becomes Toeplitz and eventually Winograd convolution algorithm can be applied. The Rader method gives a permutation scheme for indices to make matrix entries cyclic:

Rader Mapping: When N is a prime number, then a set of non-zero indices $n \in \{1, 2, ..., N-1\}$ forms a group under multiplication modulo N. One consequence of number theory of such groups is that there exists an integer generator g such that $k = g^p \pmod N$ for any non-zero index k and for a unique $p \in \{0, 1, ..., N-2\}$ (forming a bijection from p to non-zero k). Similarly $n = g^{-q} \pmod N$ for any non-zero index n and for unique $q \in \{0, 1, ..., N-2\}$ also forms a bijection (where negative exponent denotes multiplicative inverse of $g^q \pmod N$). This means that we can write Y' using new indices p and q as:

$$Y'(g^p) = \sum_{q=0}^{N-2} x(g^{-q}) W^{g^{p-q}}$$

The summation above is identical to cyclic convolution of two sequences $x(g^{-q})$ and $W^{g^{p-q}}$ of length $N-1$. For the case where $N = 5$, the generator is 2:

$$Y'(2^p) = \sum_{q=0}^{3} x(2^{-q}) W^{2^{p-q}}$$

Corresponding matrix form is:

$$\begin{bmatrix} Y'(1) \\ Y'(2) \\ Y'(4) \\ Y'(3) \end{bmatrix} = \begin{bmatrix} W^1 & W^3 & W^4 & W^2 \\ W^2 & W^1 & W^3 & W^4 \\ W^4 & W^2 & W^1 & W^3 \\ W^3 & W^4 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_3 \\ x_4 \\ x_2 \end{bmatrix}$$

Now the operation has become a length-4 cyclic convolution of sequences

$$a_q = x(2^{-q}) = \{x_1,\ x_3,\ x_4,\ x_2\}$$

$$b_{p-q} = W^{2^{p-q}} = \{W^1,\ W^2,\ W^4,\ W^3\}$$

$$Y'(2^p) = \sum_{q=0}^{3} a_q \cdot b_{(p-q)\ (\mathrm{mod}\ 4)}$$

Now we apply Winograd Fast convolution algorithm. For that we are representing this sequence as polynomials:

$$ax'(z) = x_1 + x_3 z + x_4 z^2 + x_2 z^3 \qquad W'(z) = W^1 + W^2 z + W^4 z^2 + W^3 z^3$$

The circular convolution is given by: $Y'(z) = x'(z)W'(z) \pmod{z^4 - 1}$. The required values are coefficients of $Y'(z)$

Winograd algorithm:

1. Choose a polynomial $m(z)$ with degree higher than the degree of $W'(z)x'(z)$ and factor it into $k+1$ relatively prime polynomials with real coefficients, i.e., $m(z) = m^{(0)}m^{(1)}...m^{(k)}$
2. Let $M^{(i)} = m/m^{(i)}$ and use Euclidean GCD algorithm to solve following Bezout's identity and find $N^{(i)}$:

$$m^{(i)}n^{(i)} + M^{(i)}N^{(i)} = 1$$

3. For $i = 0, 1, ..., k$.compute:

$$w^{(i)} = W'(z) \bmod m^{(i)} \tag{3.1}$$

$$x^{(i)} = x'(z) \bmod m^{(i)} \tag{3.2}$$

$$y^{(i)} = w^{(i)}x^{(i)} \mod m^{(i)} \tag{3.3}$$

4. Compute $Y'(z)$ using:

$$Y'(z) = \left[\sum_{i=0}^{k} y^{(i)}N^{(i)}M^{(i)}\right] \mod m(z) \tag{3.4}$$

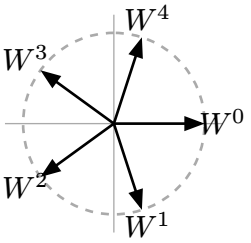
Here $m(z) = z^4 - 1 = (z-1)(z+1)(z^2+1)$. Required factors are listed in Table 3.1:

i	$m^{(i)}$	$M^{(i)}$	$x^{(i)}$	$w^{(i)}$	N_i
0	$z-1$	z^3+z^2+z+1	$x_1+x_3+x_4+x_2$	$W^1+W^2+W^4+W^3$	$1/4$
1	$z+1$	z^3-z^2+z-1	$x_1-x_3+x_4-x_2$	$W^1-W^2+W^4-W^3$	$-1/4$
2	z^2+1	z^2-1	$x_1-x_4+z(x_3-x_2)$	$W^1-W^4-z(W^3-W^2)$	$-1/2$

Table 3.1: Polynomial coefficients in winograd algorithm

We initialize some intermediate values:

$$\begin{aligned} c_{00} &= x_1 + x_4 & d_{00} &= W^1 + W^4 = 2\cos(2\pi/5) \\ c_{01} &= x_3 + x_2 & d_{01} &= W^3 + W^2 = 2\cos(4\pi/5) \\ c_{20} &= x_1 - x_4 & d_{20} &= W^1 - W^4 = -2\sin(2\pi/5)j \\ c_{21} &= x_3 - x_2 & d_{21} &= W^3 - W^2 = 2\sin(4\pi/5)j \end{aligned} \tag{3.5}$$



5 Roots of unity

and rewrite $x^{(i)}$ and $w^{(i)}$ as :

$$\begin{aligned} x^{(0)} &= c_{00} + c_{01} & w^{(0)} &= d_{00} + d_{01} = -1 \\ x^{(1)} &= c_{00} - c_{01} & w^{(1)} &= d_{00} - d_{01} = 2.236068 \\ x^{(2)} &= c_{20} + c_{21}z & w^{(2)} &= d_{20} - d_{21}z = -j1.902113 - j1.175570z \end{aligned}$$

$y^{(i)}$ are computed using Equation (3.3):

$$\begin{aligned} y^{(0)} &= -(c_{00} + c_{01}) && = -y_{00} \\ y^{(1)} &= (c_{00} - c_{01})w^{(1)} && = y_{10}w^{(1)} \\ y^{(2)} &= c_{20}d_{20} + c_{21}d_{21} + z(c_{21}d_{20} - c_{20}d_{21}) && = y_{20} + zy_{21} \end{aligned} \tag{3.6}$$

Finally using Equation (3.4):

$$\begin{aligned} Y'(z) &= -\frac{y_{00}}{4}(z^3 + z^2 + z + 1) - \frac{y_{10}}{4}(z^3 - z^2 + z - 1) - \frac{1}{2}(y_{20} + zy_{21})(z^2 - 1) \\ &= z^3\Big[-\frac{y_{00}}{4} - \frac{y_{10}}{4} + 0 - \frac{y_{21}}{2}\Big] + z^2\Big[-\frac{y_{00}}{4} + \frac{y_{10}}{4} - \frac{y_{20}}{2} + 0\Big] + \\ &\quad z\Big[-\frac{y_{00}}{4} - \frac{y_{10}}{4} + 0 + \frac{y_{21}}{2}\Big] + \Big[-\frac{y_{00}}{4} + \frac{y_{10}}{4} + \frac{y_{20}}{2} + 0\Big] \end{aligned}$$

Entire operation can be described as matrix operations:

$$\begin{bmatrix} Y(0) \\ Y(1) \\ Y(2) \\ Y(4) \\ Y(3) \end{bmatrix} = \begin{bmatrix} x_0 \\ x_0 \\ x_0 \\ x_0 \\ x_0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & 0 & 1 \\ 0 & 1 & 1 & -1 & 0 \\ 0 & 1 & -1 & 0 & -1 \end{bmatrix} \begin{bmatrix} y_{00} \\ -y_{00}/4 \\ y'_1 \\ y'_{20} \\ y'_{21} \end{bmatrix} \tag{3.7}$$

where

$$y'_1 = y_{10}\left[\frac{w^{(1)}}{4}\right] \qquad y'_{20} = c_{20}\left[\frac{d_{20}}{2}\right] + c_{21}\left[\frac{d_{21}}{2}\right] \qquad y'_{21} = c_{21}\left[\frac{d_{20}}{2}\right] - c_{20}\left[\frac{d_{21}}{2}\right] \tag{3.8}$$

Equations (3.5), (3.6), (3.7), (3.8) are used for explicitly writing the computations and hence complexity. With this method the total number of complex additions and multiplications went

from 20 and 25 to 22 and 5 respectively (That is 44 real additions and 10 multiplications). Multiplication 1/4 is right shifts and multiplication by −1 is left as implementation specific as it requires one addition when 2′s complement is used. Multiplication by d_{20} and d_{21} needs exchange of real and imaginary components. Due to large number of dependent additions, the whole operation can be pipelined and high throughput can be achieved at the cost of extensive number of intermediate registers. Full operation is shown in Figure 3.15. It is slightly different from standard winograd graph. Some additions were paired with multiplication stage in hope to effectively utilize the delay caused by multiplier.

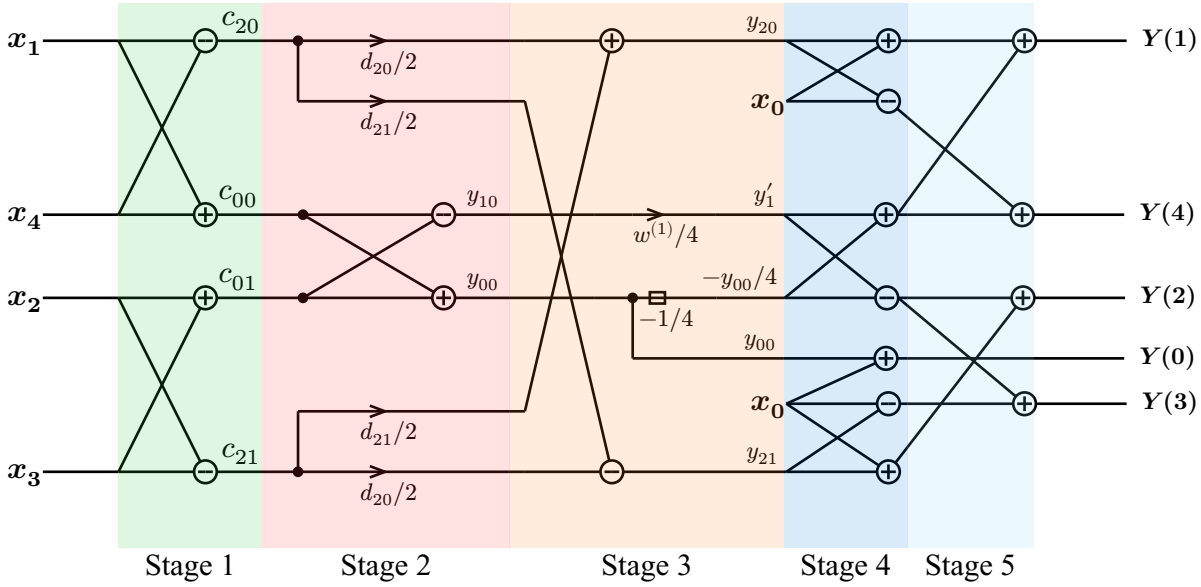


Figure 3.15: Signal flow of modified Winograd algorithm for 5 point DFT.

Moreover a direct evaluation on FPGA is not possible as it requires $DW \times 20$ I/O pins which makes it impossible to synthesize in Artix-7. Hence a wrapper is needed that takes serial input and puts them into a buffer and pass it to core compute module. To stream output serially, the output has to be split into two (either even/odd sample or chunks of $\frac{N}{2}$ samples) and be sent to outside world. This guarantees that output will never lag behind the input.

3.6.2 16 Point Winograd algorithm

The 16-point winograd can be directly obtained by considering similar permutation of inputs and outputs as discussed in the previous section. This algorithm requires 18 multiplications and 74 additions. The complete algorithm is detailed in *Section A.2: 16 point Winograd algorithm* which was obtained from [10]. Non-pipelined implementation has maximum operating frequency of 41MHz. However after a 4 stage pipelining of additions and multiplications, the operating frequency increased to 136.4MHz with total of 19 clock cycle latency.

The the output of test signals ($x_1(n) = \cos(2\pi(2/16)n) + 1$ and $x_2(n) = \cos(2\pi(4/16)n) + \cos(2\pi(6/16)n)$) are shown in Figure 3.16. The 2 colored regions highlights the inputs and their corresponding outputs. The latency is 16+2 clock cycles. It takes 2 clock cycles to finish the computations. But since the output is split to 2 paths, it takes only 8 clock cycles to completely display all outputs. Hence the system can work in real time. The output of DUT (precision is 16 bit) is compared against MATLAB in Figure 3.17 . The maximum relative error between the outputs is in the order of 10^{-4}

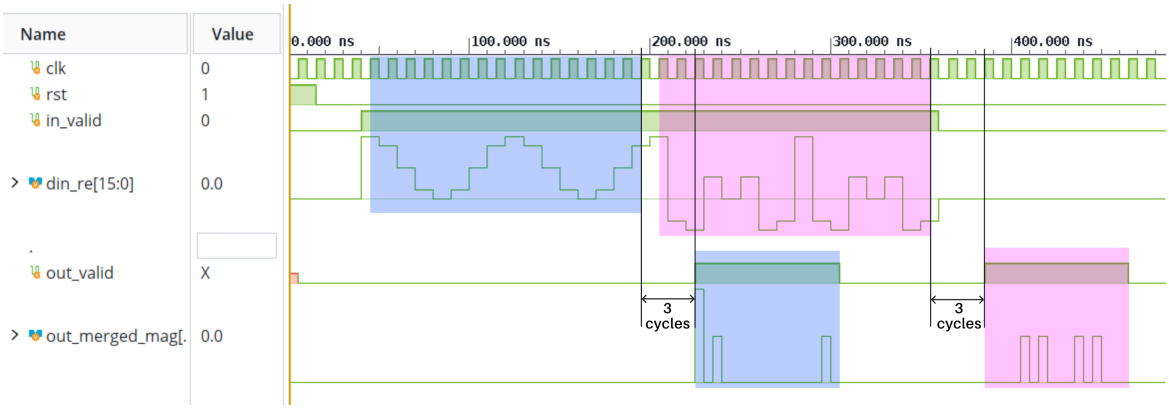


Figure 3.16: Output of 16 point 4-stage pipelined Winograd implementation for 2 consecutive inputs.

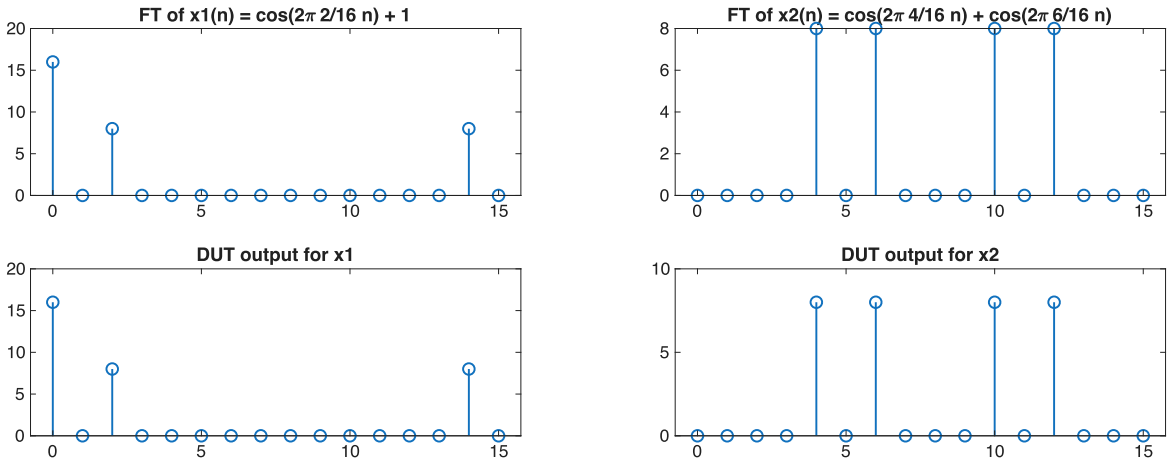


Figure 3.17: Comparison of output of FFT in MATLAB and simulation

Chapter 4:
Comparison

4.1 Comparison of 16 point transforms

To include Winograd in the comparison, case of 16 point transforms and their parameters are listed in Table 4.1.

Techniques	Total LUT	Flip-Flops	DSPs	Power (mW)	$F_{\max}$ (MHz)	Throughput (MSa/s)	Latency (cycles)
In-place	1611	555	4	104	79.2	26.41	49
R2MDC	419	645	12	187	137.1	274.2	10
R2SDC	516	327	12	154	118.3	118.3	17
Winograd	2330	2850	20	208	136.4	136.4	19

Table 4.1: Comparison of various 16 point techniques

In this case also R2MDC seems to have better edge over all other techniques. But MDC generates 2 output per cycle, hence a buffer and reordering stage is required in the subsequent sections. Winograd takes consumes significantly higher number of hardware resources in order to reduce multiplications. Perhaps my implementation might not me very efficient. Depending on pipeline depth the maximum operating frequency of winograd can be between 136.8 MHz to 142.8 MHz. However deeper pipelining increases latency.

4.2 Comparison of scalable techniques

The resource and performance of scalable techniques are shown in Figure 4.1. Same number of DSPs are required for SDC and MDC. Since In-place technique is non-pipelined, throughput is computed as $F_{\max}/(2 + \log_2(N)/2)$ as one compute frame is $2N + N\log_2(N)/2$ cycles long and no input can be supplied during this frams. From the graphs it is clear that inplace algorithm performs worse in all cases except at power consumption. Also MDC architecture is preferred in high throughput requirements (such as radar, SDR, etc.). With both pipelined architectures, the synthesis tool was able to infer BRAM blocks past a certain number of transform size. For inplace technique, performance was not calculated for transform sizes greater than 64. (At higher number of points, the design was not synthesizable due to increased number of components and complexity). The maximum frequency drops as transform size increases. This might be due to the longer paths in the FPGA required to connect extensive.

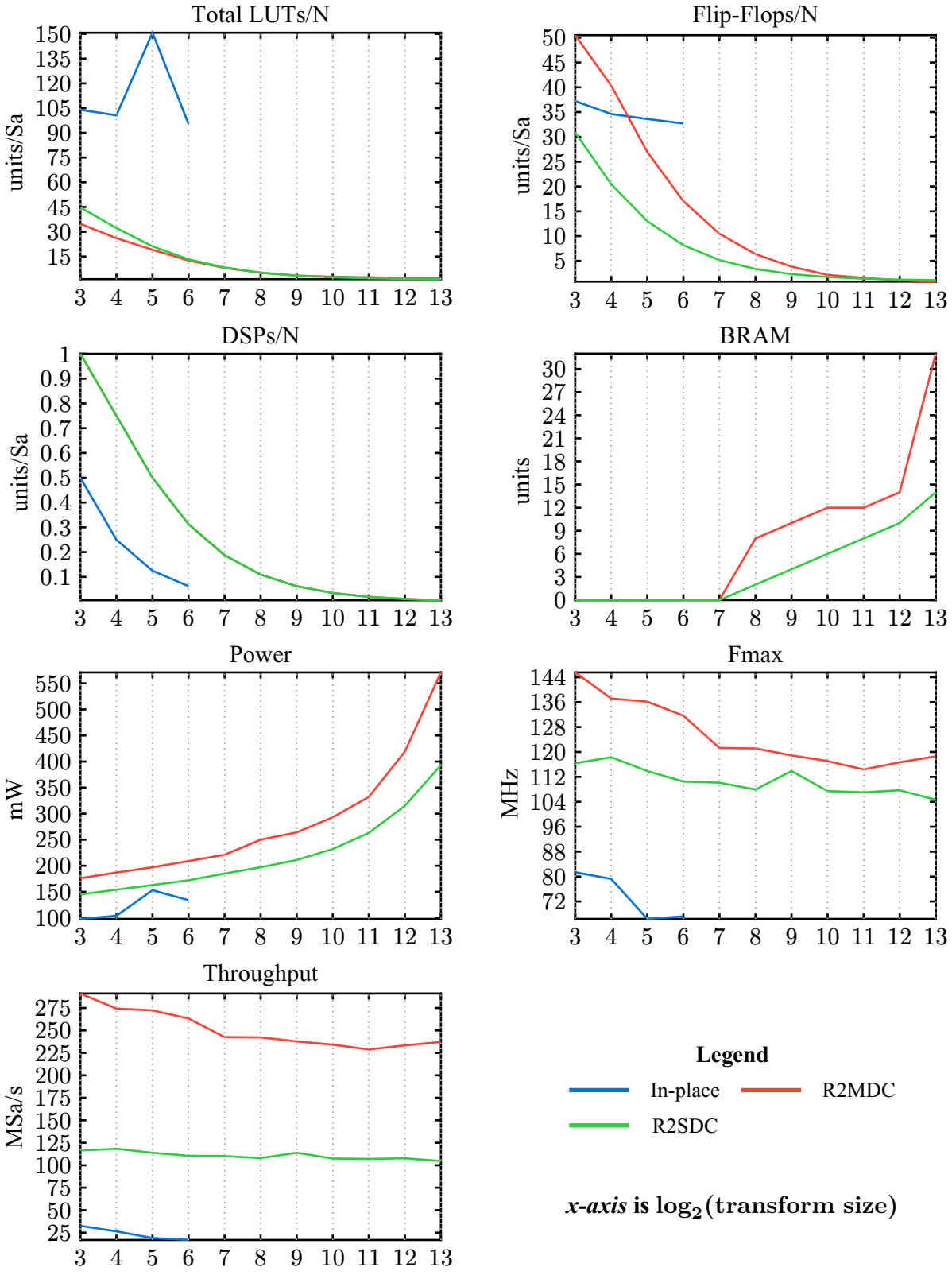


Figure 4.1: Resource and performance analysis of scalable techniques.

Chapter 5:
Conclusion

During the internship, I evaluated FFT techniques (In-place, R2MDC, R2SDC and Winograd) and their resource utilization on an Artix-7 FPGA. The analysis showed that the choice of technique depends heavily on specific performance (latency, throughput, etc.) requirement, nature of input signal (stream/available ahead of time) as well as hardware constraints (LUT, DSP, power, etc.). The performance also depends heavily on specific FPGA family, hence the comparisons I had presented are taken only in a empirical sense. Below is a quick summary of algorithms

- **In-place Radix-2 DIT**: Best suited for systems with stringent power limits and low speed requirements. It consumes the least power but has the lowest throughput due to its memory access bottleneck. The algorithm can also be mapped directly to software.
- **R2MDC**: Used for high-throughput, systems in which output data can be stored in buffer. It achieves the highest throughput by processing two samples per clock cycle. Also the outputs are in bit-reversed order.
- **R2SDC**: A middle ground for pipelined processing. It offers a steady throughput of 1 sample per cycle while using roughly half the flip-flops of MDC. Just like MDC the outputs are in bit-reversed order. Therefore further post processing is required somewhere down the processing pipeline.
- **Winograd**: Used in situation where hardware multipliers (DSP slices) are scarce. It trades multipliers for adders and LUTs. However careful implementation is required to get maximum resource efficiency.

Other methods like Good-Thomas, Prime Factor Algorithms were also looked upon and studies. Similar to Winograd, they all aims to bring down number of multipliers, because in the context of ASIC, area of multiplier and delay for processing is much higher than adders. But due to limited time resources, I did not pursue them and are not discussed here.

Chapter 6:
Future scope

The current work can be expanded in few different ways:

- Later in the course, I found that the area required during the synthesis of inplace technique can be drastically reduced by explicitly instantiating BRAM. It requires some modifications to control logic.
- In theory Winograd requires fewer number of multipliers than any Cooley-Tukey techniques. The current Winograd implementation consumes more CT techniques. This is possibly due to optimizations done by the synthesizer. Similarly other implementation can be optimized further.
- At this time, all verilog code are written to be tool agnostic and portable across different FPGA boards. This makes prototyping and learning easier as I can switch between editors and compilers. However higher performance is expected when using specific arithmetic IP core for the targetted FPGA. The performance might be drastically differnt in different FPGA families, hence various techniques can be implemented and tested in them.
- For real-valued input signals, chip area can be reduced by eliminating certain data path. Because real inputs produce symmetric DFT outputs, the hardware footprint can be pruned as the imaginary calculation pipeline and the redundant upper-frequency bins are removed. However somewhere down the pipeline, the conjugates might have to be reconstructed. But it is fairly simple as it involes changing sign of imaginary part.

References

[1] Xilinx, “7 Series FPGAs Data Sheet: Overview.” [Online]. Available: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview

[2] Xilinx, *7 Series DSP48E1 Slice*. [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

[3] Nikhilesh Bhagat, Daniel Valencia, Amirhossein Alimohammad, and Fred Harris, “High-Throughput and Compact FFT Architectures Using the Good-Thomas and Winograd Algorithms.” [Online]. Available: https://amir.sdsu.edu/Bhagat18High.pdf

[4] Wikipedia, “Cooley-Tukey FFT algorithm.” [Online]. Available: https://en.wikipedia.org/wiki/Cooley%E2%80%93Tukey_FFT_algorithm

[5] David M. Wevans, “An Improved Digit-Reversal Permutation Algorithm for the Fast Fourier and Hartley Transforms.” [Online]. Available: https://ieeexplore.ieee.org/document/1165252/

[6] Erling H Wold and Alvin M Despain, “Pipeline and Parallel-Pipeline FFT Processors for VLSI Implementations.” [Online]. Available: https://ieeexplore.ieee.org/document/1676458/

[7] Shousheng He and Mats Torkelson, “A new approach to pipeline FFT processor.” [Online]. Available: https://ieeexplore.ieee.org/document/508145

[8] Yongchul Jung, Jaechan Cho, Seongjoo Lee, and Yunho Jung, “Area-Efficient Pipelined FFT Processor for Zero-Padded Signals.” [Online]. Available: https://www.mdpi.com/2079-9292/8/12/1397

[9] P. Duhamel and M. Vetterli, *Fast Fourier Transforms: A Tutorial Review and a State of the Art*. [Online]. Available: https://dsp-book.narod.ru/DSPMW/07.PDF

[10] Henri J Nussbaumer, *Fast Fourier Transform and Convolution Algorithms*.

[11] I.J. Good, “The Relationship Between Two Fast Fourier Transforms.” [Online]. Available: https://ieeexplore.ieee.org/document/1671829

[12] K. Pagiamtzis and P.G. Gulak, “Empirical performance prediction for IFFT/FFT cores for OFDM systems-on-a-chip.” [Online]. Available: https://ieeexplore.ieee.org/document/1187288

[13] Guoan Bi and E.V. Jones, “A pipelined FFT processor for word-sequential data.” [Online]. Available: https://ieeexplore.ieee.org/document/45545

[14] Steven W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. [Online]. Available: https://www.dspguide.com/

Appendices

A.1 Source code

All source code (SystemVerilog) for the algorithms and testbenches are posted in GitHub.
`https://github.com/roopeshor/FFT-implementations`

A.2 16 point Winograd algorithm

Let $u = 2\pi/16$, $C_1 = \cos(u)$, $C_2 = \cos(2u)$, $C_3 = \cos(3u)$ $S_1 = \sin(u)$, $S_2 = \sin(2u)$, $S_3 = \sin(3u)$

Pre-additions

$$
\begin{aligned}
t_1 &= x_0 + x_8 & t_7 &= x_1 + x_9 & t_{12} &= x_5 - x_{13} & t_{17} &= t_{15} + t_{16} & t_{22} &= t_{18} + t_{20} \\
t_2 &= x_4 + x_{12} & t_8 &= x_1 - x_9 & t_{13} &= x_7 + x_{15} & t_{18} &= t_7 + t_{11} & t_{23} &= t_8 + t_{14} \\
t_3 &= x_2 + x_{10} & t_9 &= x_3 + x_{11} & t_{14} &= x_7 - x_{15} & t_{19} &= t_7 - t_{11} & t_{24} &= t_8 - t_{14} \\
t_4 &= x_2 - x_{10} & t_{10} &= x_3 - x_{11} & t_{15} &= t_1 + t_2 & t_{20} &= t_9 + t_{13} & t_{25} &= t_{10} + t_{12} \\
t_5 &= x_6 + x_{14} & t_{11} &= x_5 + x_{13} & t_{16} &= t_3 + t_5 & t_{21} &= t_9 - t_{13} & t_{26} &= t_{12} - t_{10} \\
t_6 &= x_6 - x_{14}
\end{aligned}
$$

Multiplications

$$
\begin{aligned}
m_0 &= 1(t_{17} + t_{22}) & m_5 &= C_2(t_{19} - t_{21}) & m_{10} &= j(t_{20} - t_{18}) & m_{13} &= -jS_2(t_{19} + t_{21}) \\
m_1 &= 1(t_{17} - t_{22}) & m_6 &= C_2(t_4 - t_6) & m_{11} &= j(t_5 - t_3) & m_{14} &= -jS_2(t_4 + t_6) \\
m_2 &= 1(t_{15} - t_{16}) & m_7 &= C_3(t_{24} + t_{26}) & m_{12} &= j(x_{12} - x_4) & m_{15} &= -jS_3(t_{23} + t_{25}) \\
m_3 &= 1(t_1 - t_2) & m_8 &= (C_1 + C_3)t_{24} & & & m_{16} &= j(S_3 - S_1)t_{23} \\
m_4 &= 1(x_0 - x_8) & m_9 &= (C_3 - C_1)t_{26} & & & m_{17} &= -j(S_1 + S_3)t_{25}
\end{aligned}
$$

Post-additions

$$
\begin{aligned}
s_1 &= m_3 + m_5 & s_6 &= m_4 - m_6 & s_{11} &= s_6 + s_8 & s_{16} &= m_{15} - m_{17} \\
s_2 &= m_3 - m_5 & s_7 &= m_8 - m_7 & s_{12} &= s_6 - s_8 & s_{17} &= s_{13} + s_{15} \\
s_3 &= m_{11} + m_{13} & s_8 &= m_9 - m_7 & s_{13} &= m_{12} + m_{14} & s_{18} &= s_{13} - s_{15} \\
s_4 &= m_{13} - m_{11} & s_9 &= s_5 + s_7 & s_{14} &= m_{12} - m_{14} & s_{19} &= s_{14} + s_{16} \\
s_5 &= m_4 + m_6 & s_{10} &= s_5 - s_7 & s_{15} &= m_{15} + m_{16} & s_{20} &= s_{14} - s_{16}
\end{aligned}
$$

Output Mapping (Final output)

$$
\begin{aligned}
X_0 &= m_0 & X_4 &= m_2 + m_{10} & X_8 &= m_1 & X_{12} &= m_2 - m_{10} \\
X_1 &= s_9 + s_{17} & X_5 &= s_{11} + s_{19} & X_9 &= s_{10} + s_{18} & X_{13} &= s_{12} + s_{20} \\
X_2 &= s_1 + s_3 & X_6 &= s_2 + s_4 & X_{10} &= s_2 - s_4 & X_{14} &= s_1 - s_3 \\
X_3 &= s_{12} - s_{20} & X_7 &= s_{10} - s_{18} & X_{11} &= s_{11} - s_{19} & X_{15} &= s_9 - s_{17}
\end{aligned}
$$